

Homework 3: High-Level Synthesis

2.1 Static HLS

2.1.1 Kernel 1

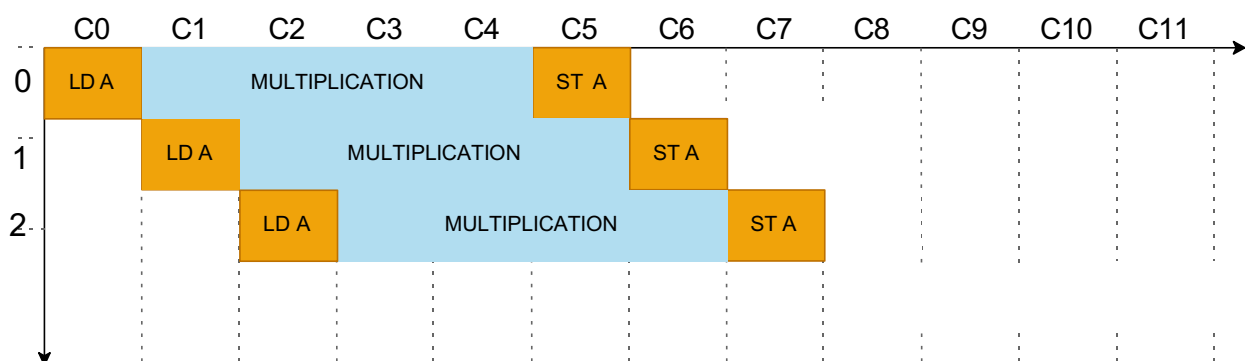
Code Modification:

```

1 void kernel1(int array[ARRAY_SIZE]) {
2   #pragma HLS array_partition variable=array type=cyclic factor=2 dim=1
3
4   loop: for(int i = 0; i < ARRAY_SIZE; i++) {
5     #pragma HLS pipeline II=1 style=stp
6       array[i] = array[i] * 5;
7   }
8 }

```

Scheduling Template:



Performance			Resources Required				
Latency (cycles)		II	Adders	Multipliers	Ports		
Per iter.	Total				array	-	-
6	ARRAY_SIZE + 5	1	0	1	2	-	-

Explanation

- Each iteration performs one load, one integer multiplication, and one store. Since an integer load/store takes 1 cycle and an integer multiplication takes 4 cycles, the latency of one iteration is: $1 + 4 + 1 = 6$ cycles

2. With loop pipelining, a new iteration can be started every cycle, so the target initiation interval is: $II = 1$
3. However, because array is originally mapped to a single-port BRAM, a read from a later iteration may conflict with a write from an earlier iteration. In the steady state, the conflicting accesses are: LD array[i], ST array[i - 5] These two indices differ by 5. Since 5 is odd, cyclic partitioning with factor=2 maps them to different memory banks:
 $array[k] \rightarrow \text{bank } k \bmod 2, array[k - 5] \rightarrow \text{bank } (k - 5) \bmod 2$ Because k and k - 5 have different parity, they are placed in different banks. Therefore, the read and write can occur in the same cycle without a port conflict.
4. Total cycles = $II \times \text{ARRAY_SIZE} + (\text{latency} - II)$
 $= 1 \times \text{ARRAY_SIZE} + (6 - 1)$
 $= \text{ARRAY_SIZE} + 5$
5. Downside

The optimization improves throughput by avoiding read/write port conflicts, but it increases hardware resource usage because the array is split into two memory banks. This requires more BRAM resources and additional banking/control logic.
6. I do not apply loop unrolling because the design already achieves $II=1$. If loop unrolling were applied, it could further reduce the total cycles, but only at the cost of significantly higher hardware resource usage.
7. `rewind` is omitted because there is only one execution of this loop/function in the considered task.

2.1.1.2 kernel 2

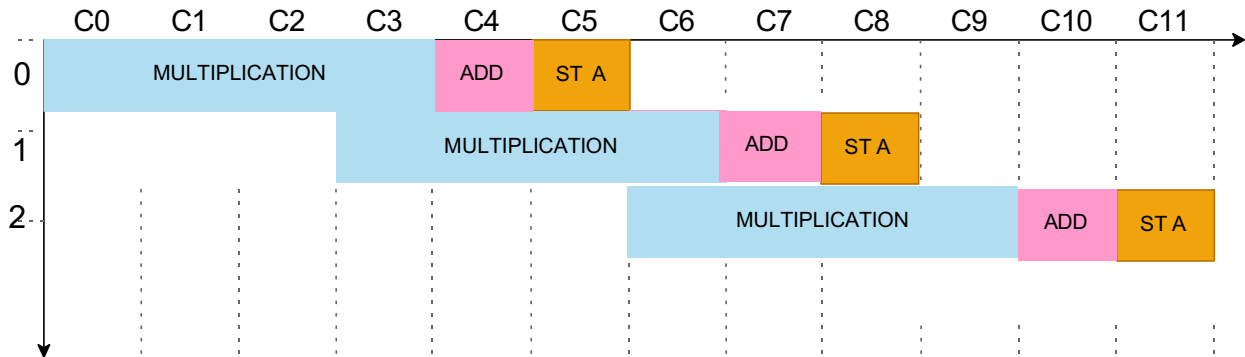
Code Modification:

```

9 void kernel2 (int array[ARRAY_SIZE]) {
10     int x0 = array[0]; // array[i - 3]
11     int x1 = array[1]; // array[i - 2]
12     int x2 = array[2]; // array[i - 1]
13
14 loop: for (int i = 3; i < ARRAY_SIZE; i++) {
15 #pragma HLS pipeline II=3 style=stp
16     int y = x2 + x1 * x0;
17     array[i] = y;
18
19     x0 = x1;
20     x1 = x2;
21     x2 = y;
22 }
```

23 }

Scheduling Template:



Performance			Resources Required				
Latency (cycles)		II	Adders	Multipliers	Ports		
Per iter.	Total				array	-	-
6	$3 \times (\text{ARRAY_SIZE} - 3) + 3$	3	1	1	1 write	-	-

Explanation

1. I rewrite the loop using three scalar registers, x0, x1, and x2, to store the previous three values of the recurrence. This avoids repeatedly loading array[i-1], array[i-2], and array[i-3] from BRAM in every iteration.
2. The latency of one iteration is: $4 + 1 + 1 = 6$ cycles
3. The loop cannot achieve $II=1$ because of a loop-carried dependency. array[i] is produced by iteration i, but it is needed as a multiplication operand by iteration i+2:

$$\text{array}[i+2] = \text{array}[i+1] + \text{array}[i] * \text{array}[i-1]$$
 Since producing array[i] requires a 4-cycle multiplication followed by a 1-cycle addition, the dependency gives: $2 \times II \geq 5$, $II \geq 3$. Therefore, the best achievable initiation interval for this design is: $II = 3$. The total number of loop iterations is: $\text{ARRAY_SIZE} - 3$. So the total latency is: $\text{Total cycles} = II \times (\text{ARRAY_SIZE} - 3) + (\text{latency} - II) = 3 \times (\text{ARRAY_SIZE} - 3) + 3$
4. Downside: this optimization uses extra scalar registers and changes the code structure. However, it significantly reduces BRAM pressure because the loop no longer performs three array reads per iteration.

2.1.3 kernel 3

Code Modification:

```

24 void kernel3(float hist[ARRAY_SIZE], float weight[ARRAY_SIZE],
25             int index[ARRAY_SIZE]) {
26
27 loop: for (int i = 0; i < ARRAY_SIZE; ++i) {
28 #pragma HLS pipeline II=5 style=stp
29     int idx = index[i];

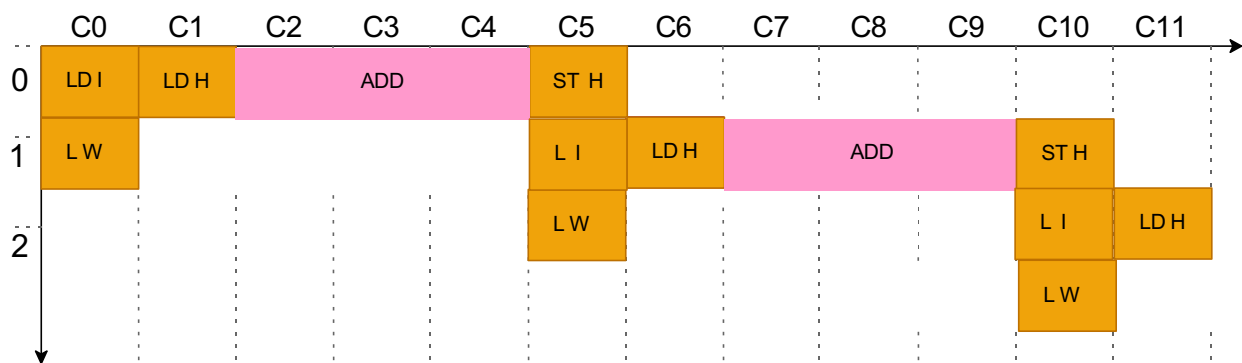
```

```

30     float w = weight[i];
31     float h = hist[idx];
32
33     hist[idx] = h + w;
34 }
35 }

```

Scheduling Template:



Performance			Resources Required				
Latency (cycles)		II	Float Adders	Multipliers	Ports		
Per iter.	Total				hist	index	weight
6	$5 \times \text{ARRAY_SIZE} + 1$	5	1	0	1 read/w rite port	1 read	1 read

Explanation

1. The latency of one iteration is $1 + 1 + 3 + 1 = 6$ cycles
2. The loop cannot safely achieve $II=1$ because there may be a RAW dependency through hist. In the worst case: $\text{index}[i] == \text{index}[i+1]$ Then iteration $i+1$ must read the updated $\text{hist}[\text{index}[i]]$, not the old value. Therefore, the next iteration cannot load the same histogram entry until the previous store is complete. Since LD hist happens one cycle after the iteration starts, and the previous store happens at cycle 5, the next iteration can start at cycle 5. Therefore: $II = 5$ Total cycles = $II \times \text{ARRAY_SIZE} + (\text{latency} - II) = 5 \times \text{ARRAY_SIZE} + 1$
3. Downside: this is a conservative schedule. If the indices are known to be independent, a false-dependency hint could allow a smaller II, but in the general case $\text{index}[i]$ and $\text{index}[i+1]$ may refer to the same histogram entry, so ignoring the dependency could produce incorrect results.

2.1.4 kernel 4 (suppose: $\text{offset} < \text{ARRAY_SIZE} - 2$)

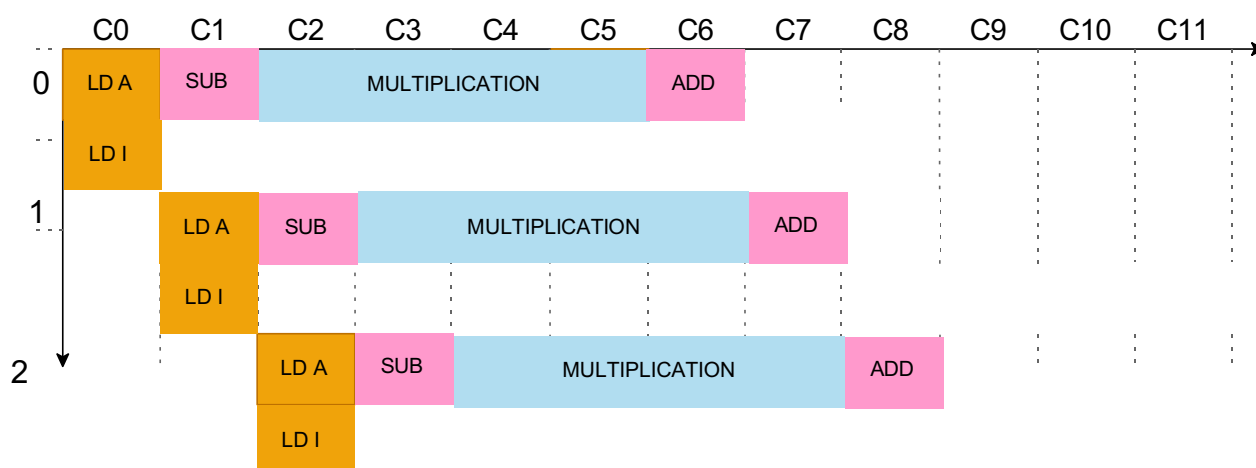
Code Modification:

```

36 void kernel4(int array[ARRAY_SIZE], int index[ARRAY_SIZE], int offset) {
37     int acc = array[offset];
38     int curr = array[offset + 1];
39
40 loop: for (int i = offset + 1; i < ARRAY_SIZE - 1; ++i) {
41 #pragma HLS pipeline II=1 style=stp
42     int idx = index[i];
43     int next = array[i + 1];
44
45     int diff = next - curr;
46     int prod = idx * diff;
47
48     acc = acc + prod;
49     curr = next;
50 }
51
52 array[offset] = acc;
53 }

```

Scheduling Template:



Performance			Resources Required				
Latency (cycles)		II	Adders/ Subtrac tor	Multipliers	Ports		
Per iter.	Total				array	index	-
7	ARRAY_SIZE - offset + 4	1	2	1	1 read/wr ite port	1 read	

Explanation

1. The latency of one iteration is: $1 + 1 + 4 + 1 = 7$ cycles
2. The loop can achieve: $II = 1$ because each iteration only needs one read from index and one read from array, and one pipelined multiplier is enough to start a new multiplication every cycle. The number of loop iterations is: $ARRAY_SIZE - offset - 2$ so the total latency is: $Total\ cycles = II \times (ARRAY_SIZE - offset - 2) + (latency - II) = ARRAY_SIZE - offset + 4$
3. Downside: this optimization uses extra scalar registers and changes the code structure. Further reducing total cycles would require loop unrolling, but that would need more multipliers, adders, memory ports, and control logic.

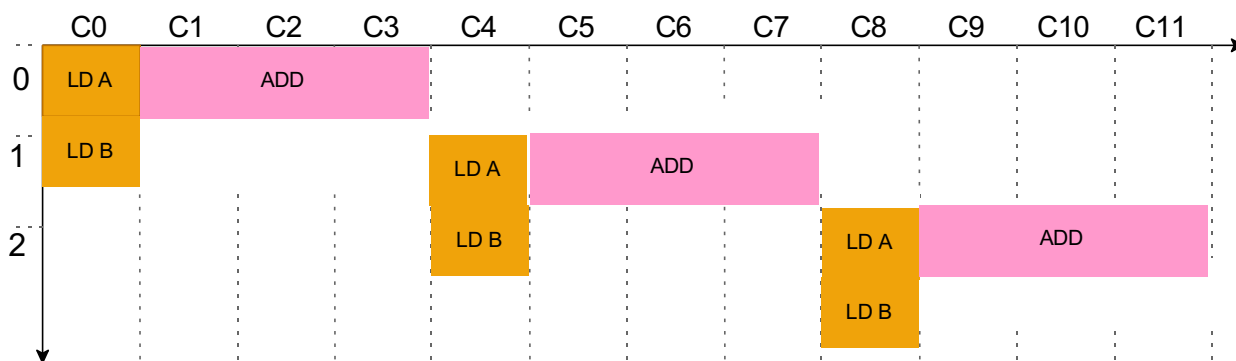
2.1.5 Kernel 5

Code Modification:

```

54 float kernel5(float bound, float a[ARRAY_SIZE], float b[ARRAY_SIZE]) {
55     int i = 0;
56     float sum = 0;
57
58 loop: while (sum < bound && i < ARRAY_SIZE) {
59 #pragma HLS pipeline II=4 style=stp
60     sum = a[i] + b[i];
61     i++;
62 }
63
64     return sum;
65 }
```

Scheduling Template:



Performance			Resources Required				
Latency (cycles)		II	Float Adders	Multipliers	Ports		
Per iter.	Total				a	b	-
4	4 × K, worst case: 4 × ARRAY_SIZE	4	1	0	1	1	

Explanation

1. The latency of one iteration is: 1 + 3 = 4 cycles
2. The loop cannot safely achieve II=1 because the loop exit condition depends on the value of sum, which is produced inside the previous iteration: while (sum < bound && i < ARRAY_SIZE) Therefore, the next iteration cannot be safely started until the previous sum is computed and the condition is known. This gives: II = 4
3. The total number of iterations is runtime-dependent. In the worst case, the loop runs ARRAY_SIZE times: Total cycles = 4 × ARRAY_SIZE
4. Downside: this loop is difficult to optimize in Static HLS because its exit condition depends on runtime data. Memory partitioning does not help much, since a and b already have separate ports. Loop unrolling with `skip_exit_check` would be unsafe, because it may execute iterations that should not occur when `sum >= bound`. Correct unrolling with explicit exit checks would preserve functionality, but it would not remove the loop-carried control dependence through `sum`, so the performance benefit would be limited. Dynamic HLS could improve average performance by handling the runtime-dependent exit condition more flexibly.

2.2 Dynamic HLS

A static non-speculative HLS implementation must wait until the previous iteration has loaded `a[i]` and `b[i]`, completed the floating-point addition, and evaluated `sum < bound` before it can safely start the next iteration. With the fixed latencies in the homework, this gives about 4 cycles per executed iteration.

A Dynamic HLS implementation replaces the fixed centralized FSM schedule with dataflow-style execution: operations fire when their data and control tokens are available. However, for this particular kernel, if all operation latencies are fixed and no speculation is allowed, the improvement is limited because the dependence through `sum` is real.

A significant improvement is possible if the dynamic implementation supports speculation on the loop continuation. For example, suppose `bound` is large and, for most early elements, `a[i] + b[i] < bound`, so the loop usually continues for many iterations. A speculative Dynamic HLS circuit can predict that the loop continues and start the loads/addition for iteration `i+1` before the branch from iteration `i` is

fully resolved. Since the prediction is usually correct for this distribution, the datapath stays busy and the effective throughput approaches one new iteration per cycle after filling the pipeline. If the prediction is wrong, the extra work is squashed. This is relatively cheap here because the loop body only reads from *a* and *b* and updates a scalar sum.

In contrast, if *bound* is small and the loop usually exits after one or two iterations, speculation gives little benefit because there are too few iterations to amortize the overhead. Dynamic HLS also costs extra hardware, such as handshake/control logic, buffers, and possibly speculation/squash logic.